

Final de Arquitectura del 6/12/2024 primera mesa

1)

```
CMPI X0, #50
B.LT E1
LSL X10, X0, 3
ADD X4, X10, X2
LDUR X3 [X3, #0]
LDUR X4 [X4, #0]
ADD X3, X3, X4
E1: CMPI X0 #100
B.EQ E2
ADD X4, X10, X5
STUR X3[X4, #0]
E2: ADDI X0, X0, #1
```

a) Completar la tabla de dependencias

b) Mostrar el orden de ejecución con procesador LEGv8 con stall con X0 = 20

c) Mostrar el orden de ejecución con procesador LEGv8 con Forwarding Stall con X0 = 100

2)

Considere el siguiente código en el que se cumplen las siguientes características

- En el caso de un excepción por interrupción externa (IRQ) el código deberá ejecutar la ISR alojada en "isr_prco" una vez que se retorne deberá retornar al flujo original del programa
- En caso de un opcode invalido se deberá reemplazar el contenido de la memoria que contiene la instrucción corrompida por 0x8B1F03FF luego se debe forzar la ejecución de este nuevo opcode
- Cualquier otra excepción deberá quedar atrapado en un lazo infinito dentro del vector de excepciones

a) completar el código

```
exc_vector: mrs X9, _____
            subis XZR, X9, _____
            b.ne jmp1
```

```
question1: _____ isr_proc
            eret
```

```
jmp1:      subis XZR, X9, 0x02
            b.ne exc_trap
            movz X9, _____, lsl #16
            movk X9, #0x03FF, lsl #0
            mrs _____, s2_0_c1_c0_0
```

```

        sturw _____ [X10, #0]
        _____ X10
exc_trap b exc_trap

```

b) Seleccionar la opción correcta

1) Si se corrompe la posición de question1 generando un opcode invalido

- i) Ante un IRQ queda en un bucle infinito
- ii) Ante un Opcode Invalido queda en un bucle infinito
- iii) Impredecible
- iv) No se realiza ninguna acción porque ya está en el vector de excepciones y retorna normalmente
- v) En cualquier caso se reemplaza el opcode invalido y se retorna al flujo original del programa previo a la primera interrupción
- vi) Ninguna de las anteriores

2) Ante la ocurrencia de un Opcode Invalido

- i) No es posible determinar la direccion de retorno
- ii) Siempre se queda atrapada en “exc_trap”
- iii) Siempre retorna a la instrucción que generó la excepción +4
- iv) Siempre retorna a la instrucción que generó la excepción
- v) Ninguna

3) Predictor global de 3 bits, GR inicial = 010

```

1> ADD 0X, XZR, XZR
2> ADD X10, XZR, XZR
3> E0: LSL X3, X0, #63
4> CBZ X3 E1
5> ADDI X10, X10, #1
6> E1: CBNZ X3 E2
7> SUBI X10, X10, #1
8> E2: ADD X0, X0, #1
9> CMP X0, #100
10> B.LT E0

```

Tabla PHT

DIR	Contenido Antes del Código	Contenido Después del Código
000	01	

001	11	
010	10	
011	01	
100	00	
101	00	
110	01	
111	10	

a) Completar con la predicción que se realiza en la primera iteración de las instrucciones

4> 6> 10>

b) Completar DC con el contenido de la PHT luego de la primera iteración

4) Completar el siguiente clock de esto

Mostrar siguiente clk

Instruction	Iteration	Instruction status		
		Issue	Execute	Write result
LSL X5, X0, #3		☒	☐	☐
ADD X4, X5, X1		☒	☐	☐
ADD X3, X5, X2		☒	☐	☐
LDUR X3, [X3, #0]		☒	☐	☐
LDUR X4, [X4, #0]		☐	☐	☐
ADD X3, X3, X4		☐	☐	☐
SUBIS X0, X0, #1		☐	☐	☐
B.NE L		☐	☐	☐
		☐	☐	☐
		☐	☐	☐

Hardware	
Issue	4 instructions
Load	3 RS / 4 clk
Store	3 RS / 4 clk
ALU int	2 UF - 4 RS / 1 clk
Mult int	2 UF - 4 RS / 2 clk

Name	Reservation stations						
	Busy	Op	Vj	Vk	Qj	Qk	A
Load 1	☐	load	-	-	ALU int 3	0	#0
	☐						
	☐						
	☐						
ALU int 1	☐	lsr	X0	3	0	0	
ALU int 2	☐	add	-	X1	ALU int 1	0	
ALU int 3	☐	add	-	X2	ALU int 1	0	
	☐						
	☐						
	☐						
	☐						

Register Status									
	D0	D1	D2	D3	D4	D5	D6	D7	D8
Qi				X3	X4	X5	X6	X7	X8
Qi	X0	X1	X2	load 1	ALU int 2	ALU int 1			